

Fine-grained Role-Based Access Control in Data Lakes by Encryption

Object Store Wrapper Service (OSWS)

Andreas Severin Hauch Trøstrup ○ Lucas Frey Torres Hanson

IT University of Copenhagen

Supervisor: Martin Hentschel

June 11th 2026

Agenda

• Introduction	3
• Related Work	4
• System Design	5
▶ Architecture Overview	6
▶ Design Choices	7
▶ Examples	11
• Methodology	13
▶ Micro-benchmarks	14
▶ E2E-benchmarks	15
• Results	16
• Discussion	17
▶ Limitations and future work considerations	18
• Conclusion	19

Introduction

- Compute and storage are separate in data lakes.
- Object Store only understands *files* $\Rightarrow$ coarse access control

Option 1: Cloud Platform

- Fine-grained access control
- Vendor lock-in
 - “fully managed all-in-one cloud platform”

Option 2: Vended Credentials

- File-level access control
 - Coarse access control
- Use preferred query engine

Solution

- OSWS
 - It decrypts columns based on authorization

Related Work

- “Membrane: A Cryptographic Access Control System for Data Lakes”
 - Cell-level access control
 - Pull the whole file, but operate only on the view
 - Client is responsible for providing keys/views
- “One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale” (Uber paper)
 - Uses standard Parquet files and PME
 - Uses a standard KMS for cryptographic keys
 - Client uses Parquet library extension to read and write

Them vs OSWS

- Them: Reader/writer needs to be modified
- OSWS: Want to ensure no reader/writer modifications
 - Previously: Wanted option for local decryption

System Design

- Wrap S3-compatible Object Store
- Provide S3-compatible API
- Enforce column-level Role-Based Access Control with Parquet Modular Encryption
- DuckDB, Spark, Python – should be able to use OSWS with no modifications

System Design: Architecture Overview

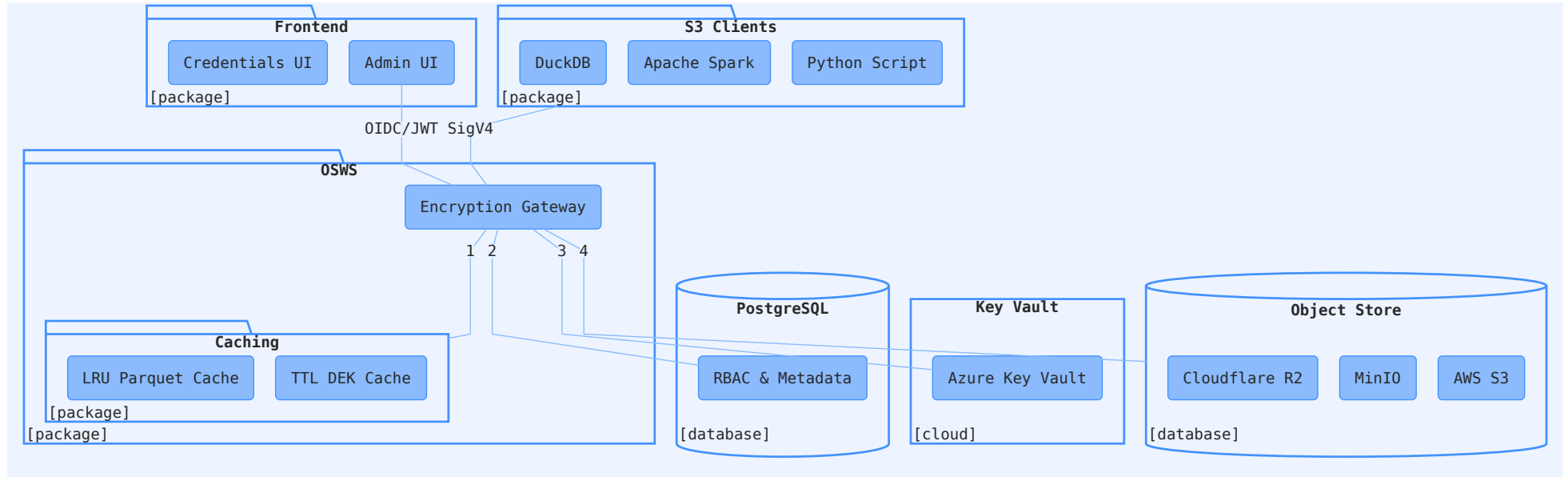


Figure 1: Architecture of OSWS showing components, external services, and communication paths
1 referencing looking up cached DEKs and Parquet files | 2 referencing RBAC calls | 3 referencing DEK wrap or unwrap | 4 referencing S3 API calls

System Design: Design Choices - Database Design + RBAC

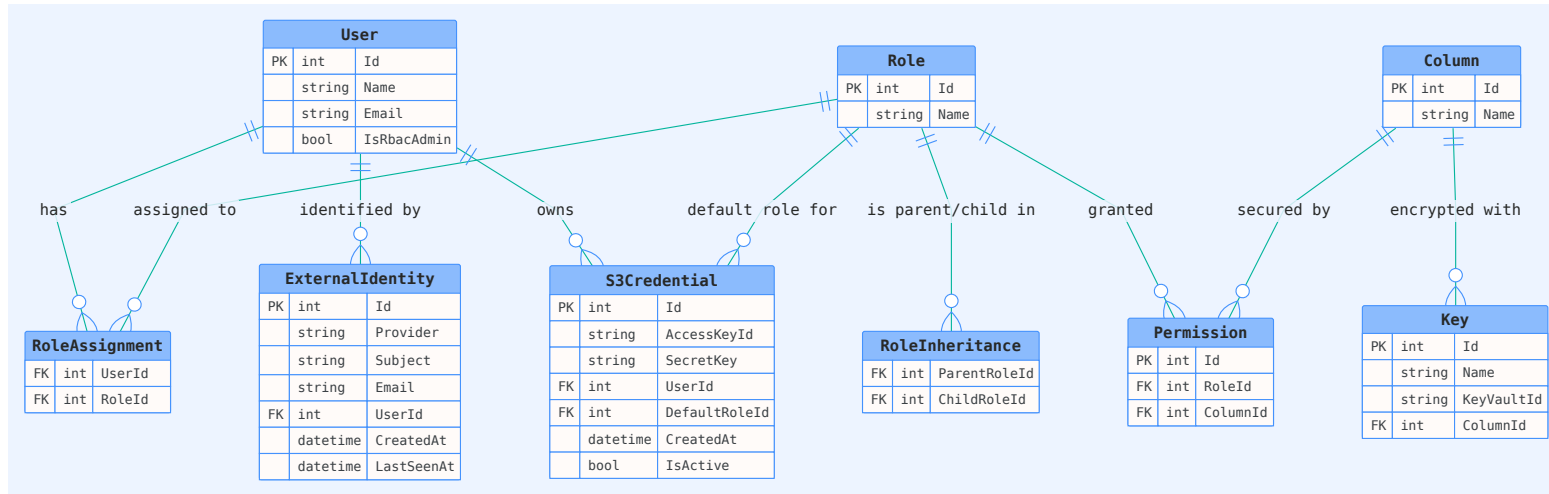


Figure 2: Entity Relationship Diagram showing the design of the database

- S3 API Credentials, Identity- & RBAC-metadata

System Design: RBAC Example

- Many-to-many: User $\leftrightarrow$ RoleAssignment $\leftrightarrow$ Role
- Many-to-many: Role $\leftrightarrow$ Permission $\leftrightarrow$ Column
- Role hierarchy via recursive SQL (self-referential RoleInheritance table)

Example:

Role *Analyst* $\rightarrow$ direct access to columns: name, age

Role *Admin* $\rightarrow$ direct access to column: ssn

Admin inherits *Analyst* $\rightarrow$ Admin sees all columns.

System Design: Design Choices - DEK Cache

- Store unwrapped DEKs in in-memory cache
 - Results in less KV calls and less latency
- The unwrapped DEK gets stored with a unique identifier which also references the KEK identifier
- Uses predefined TTL

System Design: Design Choices - Envelope Encryption

- Original idea did not include envelope encryption
- KV does not allow for fetching cryptographic keys
- Decrypting columns inside KV is extremely slow
- Allows for caching unwrapped DEKs

System Design: Walkthrough Examples - Encryption Flow

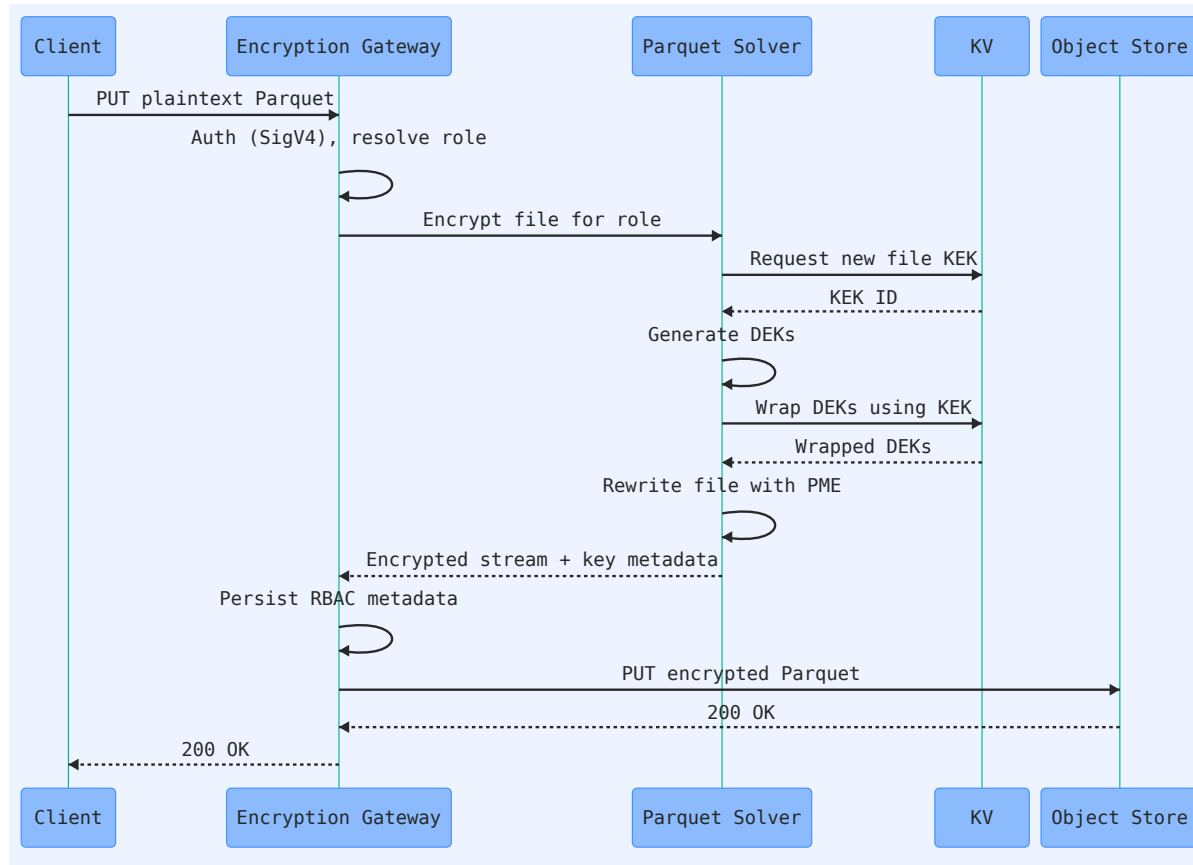


Figure 3: Encryption flow of OSWS

System Design: Walkthrough Examples - Decryption Flow

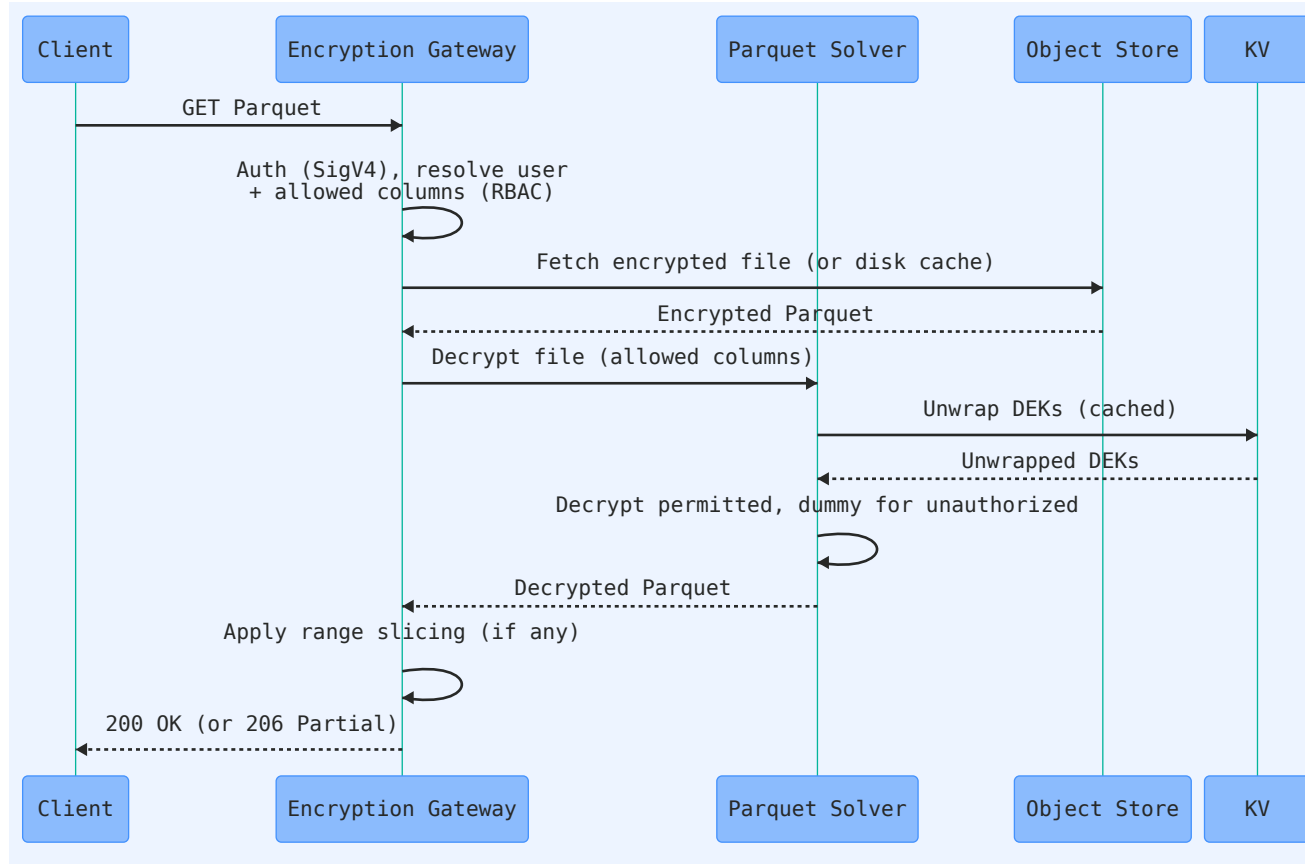


Figure 4: Decryption flow of OSWS

Methodology

- All benchmarks run on Digital Ocean VM (8 Core AMD, 16GB DRAM)
Digital Ocean Space
- Micro-benchmarks
 1. Permission Service
 2. Permission Hierarchy
 3. Unwrapping
 4. Decryption
- E2E-benchmarks
 1. S3
 2. OSWS +encryption +cache
 3. OSWS +encryption -file cache
 4. OSWS +encryption -DEK cache
 5. OSWS -encryption

Category	Benchmark	Configuration	Measurements
E2E	S3 (Spaces)	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption +caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS +encryption -file caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS +encryption -DEK caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS -encryption	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
Micro	Permission Service	No. flat hierarchy roles: 4, 64, 256	Latency
Micro	Permission Hierarchy	No. hierarchical depth roles: 0, 4, 16, 64	Latency
Micro	Unwrapping	Key size: 128, 192, 256 bits	Latency
Micro	Decryption	No. rows: 1,000, 10,000, 250,000, 1,000,000, 2,000,000 (50 columns)	Latency

Table 1: Main configurations of benchmarks that are to be run

Results: Micro-Benchmarks

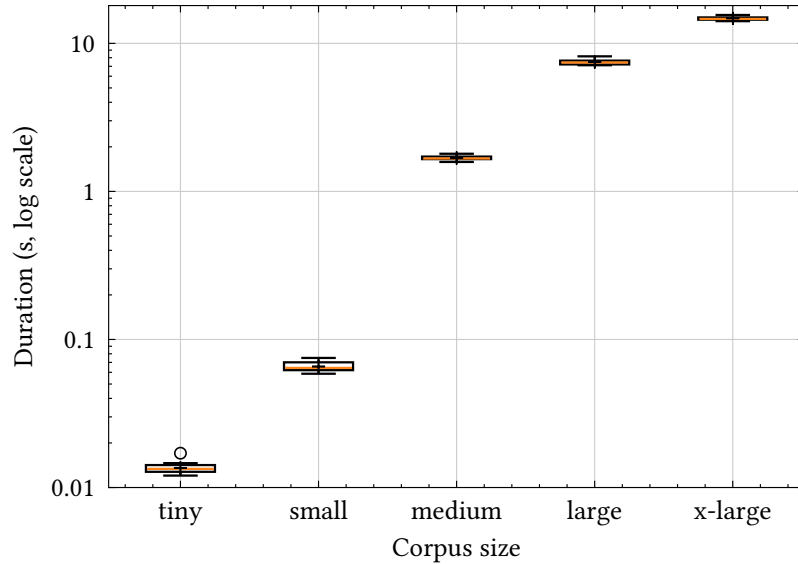


Figure 1: Decryption boxplot distribution per parameter (logarithmic y axis)

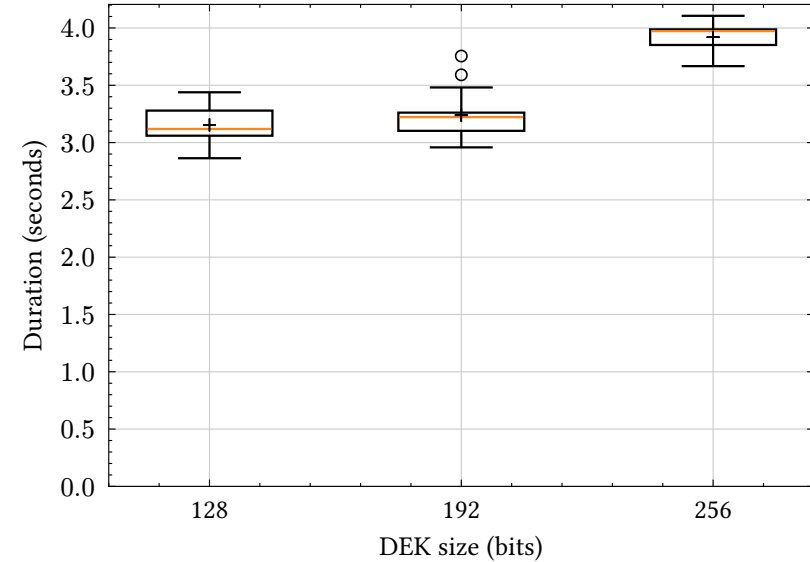


Figure 2: KeyUnwrap box-plot distribution per parameter set (tiny corpus only; linear y axis)

- Roles had minor contribution to latency ($\sim 3 - 30\text{ms}$)
- Main Bottleneck
 - DEK unwrap (fixed KV cost)
 - Decryption (scales with file size).

Results: E2E Latency

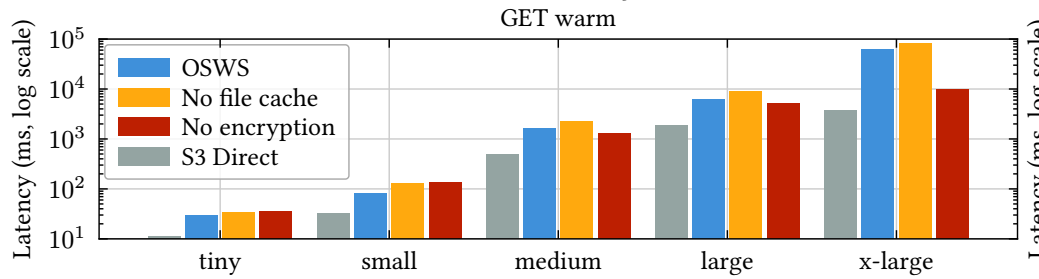


Figure 3: Median GET latency on warm cache

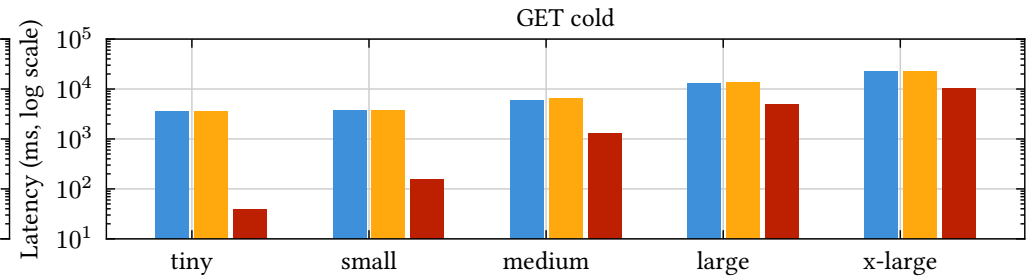


Figure 4: Median GET latency on cold cache

- Tiny: overhead high, but absolute latency low (30 ms = practical)
- Large (500 MB+): timeouts without DEK cache → needs redesign

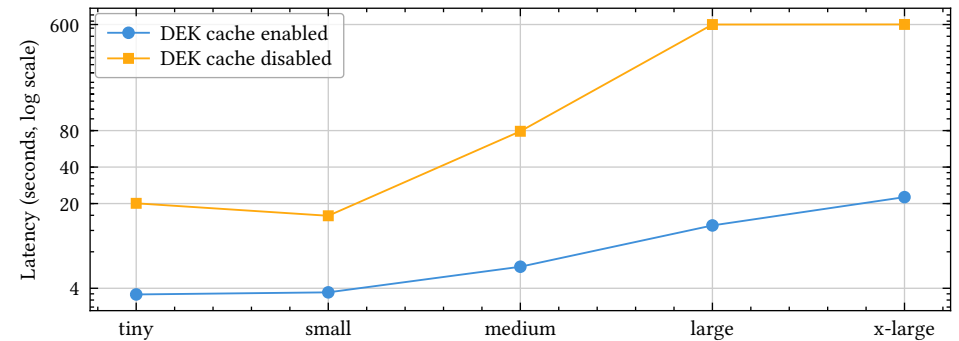


Figure 5: DEK cache impact on median cold GET latency. (logarithmic scale y axis)

Discussion: Limitations of the Current Design

The parquet file is copied instead of modified

- Parquet Sharp adds its own metadata
 - Client: written file $\neq$ read file
- Modified metadata due to PME and Parquet Sharp
- Full rewrite of the Parquet File
- Parquet Sharp + PME results in full rewrite on decryption

Parquet Sharp + PME = full-file rewrite

OSWS must COPY the **entire** Parquet file, then encrypt/decrypt every column sequentially.

- New file $\neq$ original (different writer)
- Metadata changes $\rightarrow$ file size changes
- Range requests: must fetch the entire file first

- Even unauthorized columns copied as dummy data

PME trust model: **client** handles crypto, doesn't trust storage.

OSWS needs: **server-side** crypto, client trusts the proxy.

PME was the wrong tool for this job.

Discussion: Proposed Redesign

1. Internal Metadata Store In PostgreSQL — not inside the Parquet file.

- Store wrapped DEKs, KEK ref, column offsets
- Enables range requests without full-file fetch
- Files unchanged → DuckLake/Snowflake support
- Range requests work (fetch + decrypt only needed)
- Async key pre-fetch → KV off critical path
- Production-viable performance

2. Custom AES-CTR Modifier Replace Parquet Sharp entirely.

- Length-preserving → file size never changes
- In-place decryption, no file copy
- Embarrassingly parallel row-groups

Conclusion

OSWS demonstrates feasibility of FGRBAC in data lakes by encryption

Objectives

1.  Create a POC of OSWS using an underlying S3-compatible Object Store.
2.  Ensure OSWS enforces access control using RBAC and encryption.
3.  Make it compatible with “fully managed all-in-one cloud platforms”.
4.  Make it compatible with “external query engines”.
5.  Benchmark OSWS to measure the latency it adds.
6.  Reflect on design decisions and which alternatives would have been better.

Correct Design Decisions

- DEK cache
 - Small: $\sim 5.6 \times$ improvement from none to cold
 - Small: $\sim 120 \times$ improvement from cold to warm

Impractical in Current State

- System timeout on 1GB files
- 125MB Parquet file retrieval took 1.6 seconds in best case

Questions?

Results: Micro-Benchmarks - Roles

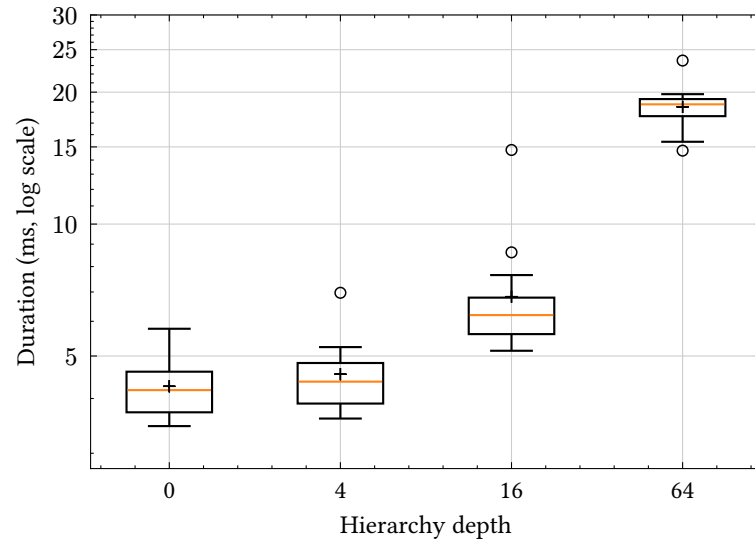


Figure 5: PermissionHierarchy box-plot distribution per parameter set (logarithmic y axis)

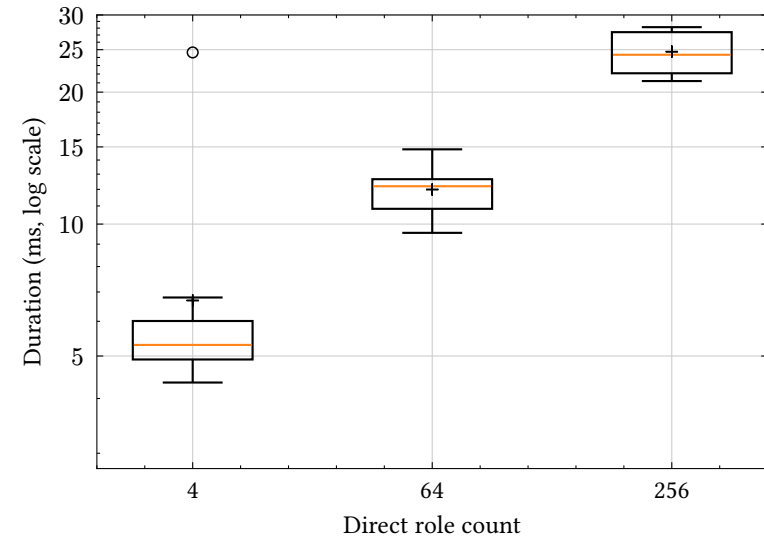


Figure 6: PermissionService box-plot distribution per parameter set (logarithmic y axis)

Results: E2E Latency - PUT

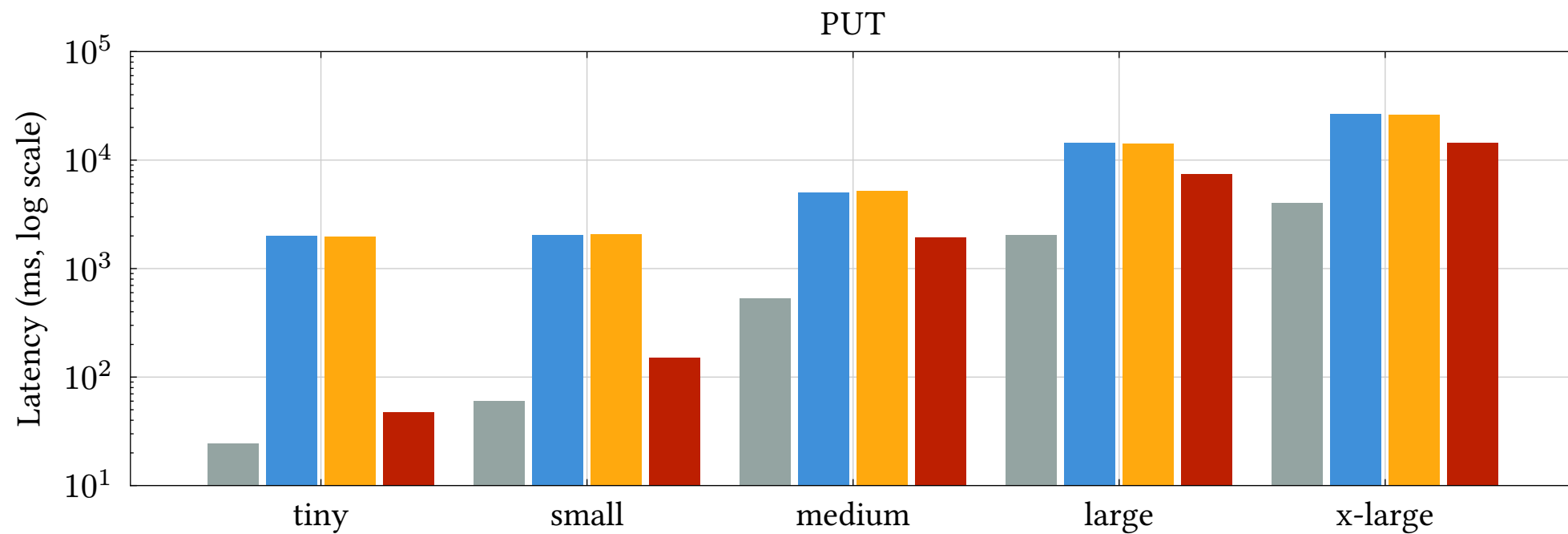


Figure 7: Median PUT latency

